Program-6

Implementation of k- nearest neighbor algorithm.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay,
)

def main():
    # Load Iris data
    iris = datasets.load_iris()
    X, y           = iris.data, iris.target
    feature_names  = iris.feature_names
    target_names   = iris.target_names

    # Train/Test split (80 / 20)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )

    # Standardize features
    scaler         = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled  = scaler.transform(X_test)

    # Fit k-NN
    k   = 5
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    #  Predict and evaluate
    y_pred     = knn.predict(X_test_scaled)
    accuracy   = accuracy_score(y_test, y_pred)
    conf_mat   = confusion_matrix(y_test, y_pred)
```

```
# Visualization ① – PCA scatter plot of ALL 150 samples
pca     = PCA(n_components=2, random_state=42)
X_vis   = pca.fit_transform(scaler.fit_transform(X))    # fit PCA on *whole* set
plt.figure(figsize=(8,6))
for i, species in enumerate(target_names):
    plt.scatter(
        X_vis[y == i, 0], X_vis[y == i, 1],
        label=species, marker="x", alpha=0.8
    )
plt.title("Iris dataset – PCA 2-D projection")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.tight_layout()
plt.savefig("iris_pca_scatter.png")  # also saves for later viewing

# Visualization ② – Confusion-matrix heat-map
plt.figure(figsize=(6,5))
ConfusionMatrixDisplay(conf_mat, display_labels=target_names).plot(values_format="d")
plt.title(f"k-NN Confusion Matrix (test set, k={k})")
plt.tight_layout()
plt.savefig("iris_knn_confusion.png")

# 8. Assemble results table and print everything
results_df = pd.DataFrame({
    "Sepal length": X_test[:,0],
    "Sepal width" : X_test[:,1],
    "Petal length": X_test[:,2],
    "Petal width" : X_test[:,3],
    "Actual"      : [target_names[i] for i in y_test],
    "Predicted"   : [target_names[i] for i in y_pred],
})

pd.set_option("display.max_rows", None)
np.set_printoptions(precision=2, suppress=True)

print("\n=== Iris Test-Set Predictions ===")
print(results_df.to_string(index=False))
print(f"\nTest-set accuracy: {accuracy*100:.2f}%")
print("\nConfusion Matrix (rows = actual, cols = predicted):\n", conf_mat)
print("\nClassification Report:\n", classification_report(
    y_test, y_pred, target_names=target_names))

print("\nFigures saved as 'iris_pca_scatter.png' and 'iris_knn_confusion.png' "\
      "in the current directory.")
```

```
if __name__ == "__main__":
    main()
```

Output-

```
=== Iris Test-Set Predictions ===
 Sepal length  Sepal width  Petal length  Petal width     Actual  Predicted
          4.4          3.0           1.3          0.2     setosa     setosa
          6.1          3.0           4.9          1.8  virginica  virginica
          4.9          2.4           3.3          1.0 versicolor versicolor
          5.0          2.3           3.3          1.0 versicolor versicolor
          4.4          3.2           1.3          0.2     setosa     setosa
          6.3          3.3           4.7          1.6 versicolor versicolor
          4.6          3.6           1.0          0.2     setosa     setosa
          5.4          3.4           1.7          0.2     setosa     setosa
          6.5          3.0           5.2          2.0  virginica  virginica
          5.4          3.0           4.5          1.5 versicolor versicolor
          7.3          2.9           6.3          1.8  virginica  virginica
          6.9          3.1           5.1          2.3  virginica  virginica
          6.5          3.0           5.8          2.2  virginica  virginica
          6.4          3.2           4.5          1.5 versicolor versicolor
          5.0          3.4           1.5          0.2     setosa     setosa
          5.0          3.3           1.4          0.2     setosa     setosa
          5.8          4.0           1.2          0.2     setosa     setosa
          5.6          2.5           3.9          1.1 versicolor versicolor
          6.1          2.9           4.7          1.4 versicolor versicolor
          6.0          3.0           4.8          1.8  virginica versicolor
          5.4          3.7           1.5          0.2     setosa     setosa
          6.7          3.1           5.6          2.4  virginica  virginica
          6.6          2.9           4.6          1.3 versicolor versicolor
          6.1          2.6           5.6          1.4  virginica versicolor
          6.4          2.8           5.6          2.2  virginica  virginica
          6.7          3.0           5.0          1.7 versicolor versicolor
          6.6          3.0           4.4          1.4 versicolor versicolor
          5.7          3.8           1.7          0.3     setosa     setosa
          6.5          3.0           5.5          1.8  virginica  virginica
          5.2          3.4           1.4          0.2     setosa     setosa

Test-set accuracy: 93.33%

Confusion Matrix (rows = actual, cols = predicted):
 [[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.83      1.00      0.91        10
   virginica       1.00      0.80      0.89        10
```

```
    accuracy                           0.93        30
   macro avg       0.94      0.93      0.93        30
weighted avg       0.94      0.93      0.93        30
```

```
Figures saved as 'iris_pca_scatter.png' and 'iris_knn_confusion.png' in the
current directory.

<Figure size 600x500 with 0 Axes>
```